

Documentación del Proyecto

Autor: César Campos - Diego Guillen

Fecha: 08/10/2025

[![Python](https://img.shields.io/badge/Python-3.9-blue.svg)](https://www.python.org/)

[![TensorFlow](https://img.shields.io/badge/TensorFlow-2.13.0-orange.svg)](https://www.tensorflow.org/)

[![License](https://img.shields.io/badge/License-GPL-blue.svg)](https://www.gnu.org/licenses/gpl.html)

Species Whisper

Species Whisper es una aplicación de inteligencia artificial para el **reconocimiento de cantos de aves** mediante grabaciones de audio.

Su principal objetivo es **funcionar de manera local**, permitiendo identificar especies incluso en **lugares sin acceso a internet**.

Objetivo del Proyecto

Desarrollar un sistema que identifique especies de aves a partir de sonidos grabados por el usuario, utilizando técnicas de **procesamiento de audio** y **modelos de aprendizaje profundo**.

La aplicación busca apoyar iniciativas de conservación, educación ambiental y monitoreo de biodiversidad en regiones rurales.

Tecnologías Principales

| Tipo | Tecnologías |

|-----|-----|

| **Framework Web** | Flask |

| **IA y Deep Learning** | TensorFlow, Keras |

| **Procesamiento de Audio** | Librosa, SoundFile, NumPy, SciPy |

| **Aprendizaje Automático** | Scikit-learn |

| **Servicios y API** | FastAPI, Uvicorn |

| **Audio y Multimedia** | FFmpeg, Pygame |

| **Documentación** | FPDF |

| **Gestión de dependencias** | requirements.in → requirements.txt |

⚙ Instalación y Ejecución

Requisitos previos

- Python 3.10 o superior

- pip y virtualenv

- FFmpeg instalado en el sistema

Configuración local

```bash

# Clonar el repositorio

git clone https://github.com/usuario/species\_whisper.git

cd species\_whisper

# Crear y activar entorno virtual

python -m venv venv

source venv/bin/activate # En Windows: venv\Scripts\activate

# Instalar dependencias

pip install -r requirements.txt

# Ejecutar la aplicación Flask

python app.py

```

La aplicación estará disponible en:

****http://127.0.0.1:5000****

Pruebas Unitarias

Las pruebas se ejecutan por módulo utilizando ****pytest****.

```
```bash
```

```
Prueba para la aplicación principal
```

```
pytest app.py
```

```
Prueba para la carga de modelos
```

```
pytest load_model.py
```

```
```
```

```
---
```

```
## Documentación de la Aplicación Flask
```

Esta aplicación permite **identificar aves mediante sus cantos** a través de una interfaz web sencilla.

1 Descripción general

- Al ejecutar `app.py`, se despliega un servidor local en `http://127.0.0.1:5000`.
- La página inicial muestra una interfaz para **cargar archivos de audio** y realizar la identificación.

2 Uso de la interfaz

1. **Cargar audio**

- Hacer clic en **"Elegir archivo"** y seleccionar el canto a identificar.

2. **Opciones después de cargar el audio**

- **Escuchar el canto** usando el reproductor de la interfaz.
- **Identificar ave**: al hacer clic, Flask llama al endpoint:

...

<http://127.0.0.1:5000/identify>


```

### 3. **\*\*Proceso interno\*\***

- El audio se guarda en `app/statics/media/audio`.

- `integrator.py` se ejecuta secuencialmente:

1. Procesar el audio

2. Cargar el modelo entrenado

3. Identificar la especie de ave

- El nombre en inglés de la especie identificada se devuelve a `app.py`, que busca los **\*\*datos correspondientes\*\*** (imagen, mapa y descripción) para mostrarlos en pantalla.

### 4. **\*\*Repetir proceso\*\***

- Para identificar otro canto, basta con hacer clic nuevamente en **\*\*"Elegir archivo"\*\*** y repetir el flujo.

### ### 3 Ejemplos visuales

- Interfaz inicial (pantalla para cargar audio):

![Interfaz inicial](reports\flask\inicio.png)

- Resultado de identificación (con especie, imagen y mapa):

![Resultado de identificación](reports\flask\prueba.png)

---

## Docker (implementación sugerida)

```
```bash
```

```
# Construir imagen
```

```
docker build -t species_whisper .
```

```
# Ejecutar contenedor
```

```
docker run -p 5000:5000 species_whisper
```

```
```
```

```

```

```
Makefile
```

El proyecto incluye un **\*\*Makefile\*\*** para automatizar las tareas más comunes del entorno de desarrollo.

### Comandos disponibles

| Comando             | Descripción                                                               |
|---------------------|---------------------------------------------------------------------------|
|                     |                                                                           |
| `make install`      | Crea un entorno virtual e instala las dependencias.                       |
| `make run`          | Ejecuta la aplicación Flask localmente.                                   |
| `make test`         | Ejecuta las pruebas unitarias (`pytest app.py` y `pytest load_model.py`). |
| `make docker-build` | Construye la imagen Docker `species_whisper`.                             |
| `make docker-run`   | Ejecuta el contenedor Docker en el puerto 5000.                           |
| `make clean`        | Elimina archivos temporales y entornos virtuales.                         |
| `make help`         | Muestra el resumen de comandos disponibles.                               |

Esto permite gestionar el ciclo completo del proyecto sin recordar los comandos largos.

Ejemplo:

```
```bash
```

```
make docker-build
```

```
make docker-run
```

```
---
```

```
---
```

```
```
```

```
Iniciar Servidor MLflow
```

Para iniciar el servidor MLflow, puedes utilizar el comando ``mlflow server`` seguido de las opciones ``--host`` y ``--port`` para especificar la dirección IP y el puerto que deseas utilizar. Por ejemplo, para iniciar el servidor en el puerto 5000, puedes utilizar el siguiente comando:

```
```bash
```

```
mlflow server --host 127.0.0.1 --port 5000
```

```
```
```

También puedes especificar un backend storage utilizando opciones como ``--backend-store-uri`` y ``--default-artifact-root``. Por ejemplo, para utilizar una base de datos PostgreSQL y almacenamiento en S3, puedes utilizar el siguiente comando:

```
```bash
```

```
mlflow server \
```

```
--backend-store-uri postgresql://user:password@localhost/mlflow \
```

```
--default-artifact-root s3://my-mlflow-bucket/ \
```

```
--host 0.0.0.0 \
```

```
--port 5000
```

```
```
```

## ## Uso del MLflow Manager

El MLflow Manager es una herramienta que te permite interactuar con el servidor MLflow y realizar tareas como registrar experimentos de entrenamiento, cargar modelos y registrar identificaciones de aves. Puedes crear un gestor con un servidor local utilizando el siguiente código:

```
```python
```

```
from mlflow_manager import MLflowModelManager
```

```
manager = MLflowModelManager(  
  
    tracking_uri="http://127.0.0.1:5000",  
  
    experiment_name="BirdWhisper"  
  
)  
  
...
```

Puedes registrar un experimento de entrenamiento utilizando el método ``log_model_training`` y cargar un modelo desde el registro utilizando el método ``load_model_from_registry``. También puedes registrar identificaciones de aves utilizando el método ``log_birdnet_experiment``.

```
...
```

Estructura del Proyecto

```
...
```


species_whisper/

├─ app.py # Aplicación Flask

├─ load_model.py # Carga y predicción del modelo de IA

├─ static/ # Recursos estáticos (iconos, CSS, JS)

├─ templates/ # Vistas HTML

├─ models/ # Modelos entrenados

├─ tests/ # Pruebas unitarias

├─ requirements.in

├─ requirements.txt

└─ README.md

...

Autores

- **Diego Guillen**

- **César Campos**

Licencia

Este proyecto está licenciado bajo la **GNU General Public License v3.0 (GPLv3)**.

Puedes usar, modificar y distribuir este software libremente bajo los términos de esta licencia,

siempre y cuando cualquier distribución o versión modificada también sea de código abierto.

Consulta el archivo LICENSE para más información.

Creditos de las imagenes

Créditos de Imágenes

Las imágenes listadas a continuación provienen de **Macaulay Library / Cornell Lab of Ornithology (vía eBird)** y son propiedad de los respectivos autores. Se usan en este proyecto con fines educativos y de investigación. Antes de cualquier uso comercial o redistribución, verificar la licencia y contactar al autor si es necesario.

- **Tinamus osgoodi** (Black Tinamou) — Brayan Coral Jaramillo — ML26233561. :contentReference[oaicite:0]{index=0}

- ****Ruddy Duck**** (**Oxyura jamaicensis**) — (ej. ML618526817) — contributor/asset en Macaulay Library.
:contentReference[oaicite:1]{index=1}

- ****Cauca Guan**** — (ej. ML22265691 / ML201224271) — asset(s) en Macaulay Library. :contentReference[oaicite:2]{index=2}

- ****Gorgeted Wood-Quail**** (**Odontophorus strophium**) — Mauricio Álvarez (ej. ML250808). :contentReference[oaicite:3]{index=3}

- ****Podiceps occipitalis**** (Silvery Grebe) — Daniel Pettersson / Pio Marshall (ej. ML144655831 / ML168457431). :contentReference[oaicite:4]{index=4}

- ****Phoenicopterus ruber**** (American/Caribbean Flamingo) — asset(es) en Macaulay Library (varios ML recientes listados). :contentReference[oaicite:5]{index=5}

- ****Sula leucogaster**** (Brown Booby / Masked Booby group) — Timothy Barksdale (ej. ML428555). :contentReference[oaicite:6]{index=6}

- ****Egretta rufescens**** (Reddish Egret) — asset(s) en Macaulay Library (ej. ML625527611). :contentReference[oaicite:7]{index=7}

- ****Vultur gryphus**** (Andean Condor) — Pablo Re / otros (ej. ML434115881). :contentReference[oaicite:8]{index=8}

- **Buteogallus solitarius** (Solitary Eagle) — asset(s) en Macaulay Library (ej. ML630019576 / ML61567901). :contentReference[oaicite:9]{index=9}

- **Bogota Rail** (*Rallus semiplumbeus*) — Lisa & Li Li (ej. ML585960821 / ML615330810). :contentReference[oaicite:10]{index=10}

- **Tolima Dove** (*Leptotila conoveri*) — Edier Rojas Parra (ej. ML598418671). :contentReference[oaicite:11]{index=11}

- **Pied Puffbird** (*Notharchus tectus*) — varios contributors (ej. ML201952871 / ML619050118 / ML201701501). :contentReference[oaicite:12]{index=12}

- **Eriocnemis isabellae** (Gorgeted Puffleg) — ProAves Colombia / otros (ej. ML265216381). :contentReference[oaicite:13]{index=13}

- **Capito hypoleucus** (White-mantled Barbet) — Juan Fernando Giraldo Lopera (ej. ML639829783). :contentReference[oaicite:14]{index=14}

- **Campephilus गयाquilensis** (Guayaquil Woodpecker) — Gregory Griffith / Chris Thomas (ej. ML436797 / ML618939526). :contentReference[oaicite:15]{index=15}

- **Ara ambiguus** (Great Green Macaw) — Hal & Kirsten Snyder / Hervé JACOB (ej. ML204445871 / ML443538601). :contentReference[oaicite:16]{index=16}

- **Apolinar's Wren** (*Cistothorus apolinari*) — varios assets; ejemplo ML613904284 / ML616340778. :contentReference[oaicite:17]{index=17}
- **Chlorospingus** (relacionado a *Chlorospingus flavopectus* / "Common Chlorospingus") — varios contributors (ej. ML132196911 / ML524469541 / ML623321090). :contentReference[oaicite:18]{index=18}
- **Hypopyrrhus pyrohypogaster** (Red-bellied Grackle) — varios assets (ej. ML277859941 / ML633531268 / ML610261738). :contentReference[oaicite:19]{index=19}

Futuro del Proyecto

- Implementar un modelo más ligero para dispositivos de bajo consumo.
- Crear una versión móvil offline.
- Añadir identificación por imágenes y sonidos combinados.

Resultados de Pruebas Unitarias

=====	test	session	starts
=====			
platform win32 -- Python 3.12.2, pytest-8.4.2, pluggy-1.6.0 -- C:\Users\CESAR CAMPOS\species_whisper\.venv\Scripts\python.exe			
cachedir: .pytest_cache			
rootdir: C:\Users\CESAR CAMPOS\species_whisper			
plugins: anyio-4.11.0			
collecting ... collected 2 items			
tests/test_app.py::test_home_route PASSED		[50%]	
tests/test_integrator.py::test_get_species_prediction PASSED		[100%]	
=====	2	passed	in 14.88s
=====			